

The Ray Tracing Sampler

Most code: Copyright ©2024–2026 Peter Behroozi

PyTorch version: Based on the Adam optimizer, Copyright ©2024 PyTorch Developers

JAX version: Based on the minimal-hmc repository, Copyright ©2024 Martin Marek

xoshiro256** RNG: based on work by David Blackman and Sebastiano Vigna

License (C and JAX versions, including examples): Apache 2.0

License (PyTorch version, including examples): PyTorch (modified BSD)

License (Autocorrelation Time Estimator): GPLv3

Science/Documentation Paper: <https://arxiv.org/abs/2510.25824>

Contents

1 Overview	1
2 PyTorch version	1
3 JAX version	5
4 C version	7
5 Examples	10
6 Autocorrelation Time Utility	10

1 Overview

The ray tracing algorithm carries the most advantages relative to existing methods when stochastic gradients are used, for example when performing Bayesian sampling of neural networks. These docs describe the C, JAX, and PyTorch reference implementations, which are intended to be transparent and correct, rather than maximally performant. Each of these implementations also includes an HMC sampler implementation for ease of comparison.

2 PyTorch version

To install, run `pip install raytrace_torch` or copy the `raytrace_torch.py` file to your source directory. The PyTorch implementation inherits from the `torch.optim` class, so for basic usage, you can just replace your normal call to create an optimizer with a call to the ray tracing package:

```

from raytrace_torch import Raytracer

#Initialize typically using a pre-optimized parameter set,
# and sample according to a loss tolerance X and timestep Y
optimizer = Raytracer(model.parameters(), loss_tolerance=X, dt=Y)

```

You can then perform your usual call to `optimizer.step()`, and the model parameters will be sampled instead of optimized.

Generally, you will want to initialize model parameters from a previously-optimized model (e.g., with Adam or similar), as ray tracing will not be as efficient at finding the model minimum as algorithms that are designed to minimize the loss as fast as possible. You will also want to adapt the internal likelihood scaling and the timestep size to your problem.

Sampling will not give as low loss values as optimization, by definition. You can tune this difference by setting your desired value for the `loss_tolerance` parameter. Internally, this scales the loss function (see the ray tracing paper) assuming that the effective number of constrained dimensions of the network is the same as the network parameter count. Since this is not always true, you may find that you need to increase the `loss_tolerance` parameter, especially if the number of samples in your training set is less than the parameter count—but generally if your network is significantly more complex than the underlying function you are trying to fit. As an alternative to the `loss_tolerance` parameter, you can set the `scale_likelihood` parameter, which scales the loss function directly. A strong sign that you need to increase the `loss_tolerance` parameter (or, equivalently, decrease the `scale_likelihood` parameter) is if your validation loss is larger than your training loss — i.e., if overfitting is occurring.

The choice of timestep size depends on the stochasticity of the gradient, with higher stochasticity (i.e., lower mini-batch sizes) needing lower timestep sizes. The default timestep size (which is used if `dt=0` or if `dt` is not provided) is about $0.03/D^{0.5}$, where D is the network parameter count. Fortunately, it's relatively easy to tell if the timestep is too large or small. If the loss is diverging (which usually occurs within a few dozen steps after starting the chain), try repeatedly lowering the timestep by an order of magnitude until it no longer diverges. Then, increase the timestep size by factors of 2 until you reach divergence again, and use the largest timestep that does not result in divergence. Readers of your results will appreciate if you check for convergence by using a smaller timestep as well (e.g., 1/5 the size) and confirming that the posteriors that you find are consistent.

The list of optional parameters to the Raytracer initialization is:

- `loss_tolerance=float`: the acceptable additional loss above the level achieved by an optimizer. As above, this value may need to be increased if your model is much more complex than the underlying data or if your training set is small. You can alternately set the `scale_likelihood=float` parameter to choose the likelihood

scaling directly. Note that, like other optimizers, this implementation assumes that the gradient is being taken of the loss function, instead of the likelihood function, and so the sign of the gradient is automatically inverted internally.

- `dt=float`: the timestep size of leapfrog integration. The corresponding step size is a factor $D^{0.5}$ larger than the timestep size, where D is the number of trainable model parameters.
- `refresh_rate=float`: the rate at which the ray tracing momentum is continuously refreshed. The default is to refresh momentum after a travel time of 1. Lower refresh rates result in less random-walk-like behavior at the cost of potentially slower mixing, and higher refresh rates have the opposite behavior. A refresh rate of 1, for example, will refresh the momentum after a travel time of 1, whereas a refresh rate of 2 will refresh the momentum after a travel time of 0.5.
- `stochastic_hmc=bool`: if set to 1, this results in HMC sampling instead of ray tracing.
- `verbose=bool`: if set to 1, prints out the parameter and momentum norms at every optimization step.

There are also several helper functions to enable Metropolis tests and more rigorous sampling (all called as, e.g., `optimizer.first_step()`):

- `first_step()`: Call before the first call to `step()` in the sampling trajectory so that the leapfrog integrator knows to perform the first half-step of integration. (Only useful if a Metropolis test is being performed).
- `final_step()`: Call before the last call to `step()` in the sampling trajectory so that the leapfrog integrator knows to perform the final half-step of integration. (Only useful if a Metropolis test is being performed).
- `reverse_momentum()`: Call after a rejected Metropolis test if you are using continuous refresh rates, to ensure reversibility. As the name describes, it will reverse the direction of propagation.
- `scale_likelihood()`: Returns the likelihood scaling factor S .
- `ln_luminosity()`: Returns the change in log luminosity since the last `first_step()` call. To perform a Metropolis test, this should be compared to the change in the scaled loss function.

You may also change parameters after the sampler is initialized (e.g., if you have started a run and realize that you need to change the timestep size, etc.).

- `set_scale_likelihood(X)`: Change the likelihood scaling to X (`float`).

- `set_dt(X)`: Change the timestep to `X` (`float`).
- `set_refresh_rate(X)`: Change the refresh rate to `X` (`float`).

The PyTorch version also supports a JAX-like interface. For example,

```
from raytrace_torch import sample_raytrace

chain, likelihood = sample_raytrace(params_init=x0, \
    log_prob_fn=lpf, n_steps=1000, n_leapfrog_steps=10, \
    step_size=0.15, refresh_rate=0.0, metro_check=1, sample_hmc=False, \
    device=None, samples_device=None, scale_likelihood=1.0)
```

The parameters are as follows:

- `params_init`: a PyTorch tensor for the starting position. The number of dimensions of the parameter space is inferred based on the number of elements in this tensor.
- `log_prob_fn`: a function that returns the log-likelihood. If you're used to calculating chi-squared values instead, the log-likelihood will be a factor of $-1/2$ times the chi-squared value. If you are computing a neural network loss, see the tips above about scaling the loss appropriately (and note that the log-likelihood should be proportional to the negative loss, so lower losses correspond to higher likelihoods—this method does not invert the sign of the gradient like the Optimizer version above). The likelihood may be stochastic (e.g., from a mini-batch of a neural network), but in this case, you may wish to disable Metropolis checks.
- `n_steps`: the number of trajectories to integrate, i.e., the number of points in the returned chain.
- `n_leapfrog_steps`: the number of integration steps per trajectory.
- `step_size`: the timestep size. The physical step size will be a factor $D^{0.5}$ larger, where D is the parameter space dimensionality.
- `refresh_rate=0`: (*optional*) the rate at which the ray tracing momentum is continuously refreshed. The default is no refresh. Lower refresh rates result in less random-walk-like behavior at the cost of potentially slower mixing, and higher refresh rates have the opposite behavior. A refresh rate of 1, for example, will refresh the momentum after a travel time of 1, whereas a refresh rate of 2 will refresh the momentum after a travel time of 0.5.
- `metro_check=1`: (*optional*) set to 1 (the default) to perform Metropolis checks and 0 to skip them.

- `sample_hmc=False`: (*optional*) set to `False` (the default) to perform ray tracing and `True` to perform HMC sampling.
- `device=None`: (*optional*) the device where parameters and gradients are stored. By default, this is the same as the device where `x0` is stored.
- `samples_device=None`: (*optional*) the device where the chain samples are stored. By default, this is the same as the device where `x0` is stored.
- `scale_likelihood=1.0`: (*optional*) the scaling applied to the likelihood function.

3 JAX version

To install, run `pip install raytrace_jax` or copy the `raytrace_jax.py` file to your source directory. It's easiest to consider an example, e.g., of sampling from a 10,000-dimensional Gaussian distribution:

```
import jax
from raytrace_jax import sample_raytrace

def gaussian(params):
    return jax.scipy.stats.norm.logpdf(params).sum()

k,nk = jax.random.split(jax.random.PRNGKey(0), 2)
x0 = jax.random.normal(nk, shape=[10000])
chain,likelihood = sample_raytrace(key=k, params_init=x0, \
    log_prob_fn=gaussian, n_steps=1000, n_leapfrog_steps=10, \
    step_size=0.15, refresh_rate=0.0, metro_check=1, sample_hmc=False)
```

The parameters are as follows:

- `key`: a JAX random key.
- `params_init`: a JAX array for the starting position. The number of dimensions of the parameter space is inferred based on the length of this array.
- `log_prob_fn`: a function that returns the log-likelihood. If you're used to calculating chi-squared values instead, the log-likelihood will be a factor of $-1/2$ times the chi-squared value. If you are computing a neural network loss, see the tips in the ray tracing paper about scaling the loss appropriately. The likelihood may be stochastic (e.g., from a mini-batch of a neural network), but in this case, you may wish to disable Metropolis checks.
- `n_steps`: the number of trajectories to integrate, i.e., the number of points in the returned chain.

- `n_leapfrog_steps`: the number of integration steps per trajectory.
- `step_size`: the timestep size. The physical step size will be a factor $D^{0.5}$ larger, where D is the parameter space dimensionality.
- `refresh_rate=0`: (*optional*) the rate at which the ray tracing momentum is continuously refreshed. The default is no refresh. Lower refresh rates result in less random-walk-like behavior at the cost of potentially slower mixing, and higher refresh rates have the opposite behavior. A refresh rate of 1, for example, will refresh the momentum after a travel time of 1, whereas a refresh rate of 2 will refresh the momentum after a travel time of 0.5.
- `metro_check=1`: (*optional*) set to 1 (the default) to perform Metropolis checks and 0 to skip them.
- `sample_hmc=False`: (*optional*) set to False (the default) to perform ray tracing and True to perform HMC sampling.

To perform HMC sampling, you may also import and call `sample_hamiltonian()` instead of `sample_raytrace()`; the function parameters are identical, except that the default value of `sample_hmc` is True.

If you are using ray tracing to sample a neural network, you will want to initialize model parameters from a previously-optimized model (e.g., with Adam or similar), as ray tracing will not be as efficient at finding the model minimum as algorithms that are designed to minimize the loss as fast as possible. You will also want to adapt the likelihood scaling and the timestep size to your problem, as below.

Sampling will not give as low loss values as optimization, by definition. You can tune this difference by defining an appropriate likelihood that is a scaled version of your loss function. If D is the number of dimensions, a reasonable starting scaling is $L(x) = f_{\text{loss}}(x) * D / (2f_{\text{tol}})$, where f_{tol} is your desired loss tolerance. This choice scales the loss function (see the ray tracing paper) assuming that the effective number of constrained dimensions of the network is the same as the network parameter count. Since this is not always true, you may find that you need to decrease the scaling factor ($D / (2f_{\text{tol}})$), especially if the number of samples in your training set is less than the parameter count—but generally if your network is significantly more complex than the underlying function you are trying to fit. A strong sign that you need to decrease the scaling factor is if your validation loss is larger than your training loss — i.e., if overfitting is occurring.

The choice of timestep size depends on the geometry of the likelihood function and the stochasticity of the gradient, with higher stochasticity (i.e., lower mini-batch sizes) needing lower timestep sizes. For neural networks, a recommended starting timestep size is about $0.03/D^{0.5}$. Fortunately, it's relatively easy to tell if the timestep is too large or small. If the loss is diverging (which usually occurs within a few dozen steps

after starting the chain), try repeatedly lowering the timestep by an order of magnitude until it no longer diverges. Then, increase the timestep size by factors of 2 until you reach divergence again, and use the largest timestep that does not result in divergence. Readers of your results will appreciate if you check for convergence by using a smaller timestep as well (e.g., 1/5 the size) and confirming that the posteriors that you find are consistent.

4 C version

To install, copy the `raytrace.c` and `raytrace.h` files, and link to them when you compile. The C interface defines a `raytracer` struct, which contains all the sampling parameters. You should include the `raytrace.h` header file in all code that interfaces with the sampler. To initialize a sampler, you should use the `Init_Raytracer()` call:

```
struct raytracer *Init_Raytracer(double *x, int64_t DIMS,
    int64_t trajectory_steps, double dt, double refresh_rate,
    double (*likelihood)(double *, int64_t),
    void (*gradient)(double *, double *, int64_t),
    int64_t metropolis_check, enum raytrace_integration_type itype,
    enum raytrace_mcmc_type mcmc_type);
```

The parameters are as follows:

- `double *x`: a pointer to an array of `doubles` that contain the initial starting location.
- `int64_t DIMS`: the number of dimensions D of the parameter space.
- `int64_t trajectory_steps`: the number of integration steps per trajectory.
- `double dt`: The timestep. the actual step size will be a factor $\text{DIMS}^{0.5}$ larger.
- `double refresh_rate`: the rate at which the ray tracing momentum is continuously refreshed. A refresh rate of 1, for example, will refresh the momentum after a travel time of 1, whereas a refresh rate of 2 will refresh the momentum after a travel time of 0.5. Lower refresh rates result in less random-walk-like behavior at the cost of potentially slower mixing, and higher refresh rates have the opposite behavior.
- `double (*likelihood)(double *, int64_t DIMS)`: a **pointer to a function** that accepts a location in parameter space (`double *`) and the number of dimensions (`int64_t`) of the parameter space, and returns a `double` value for the log-likelihood. If you're used to calculating chi-squared values instead, the log-likelihood will be a factor

of $-1/2$ times the chi-squared value. If you are computing a neural network loss, see the tips in the ray tracing paper about scaling the loss appropriately.

- `void (*gradient)(double *x, double *grad, int64_t DIMS)`: a pointer to a function that accepts a location in parameter space (`double *`), an array to store the gradient (`double *`), and the number of dimensions (`int64_t`) of the parameter space. It should compute the gradient (stochastic gradients are OK), and store in the supplied gradient array. If you have scaled the likelihood function above, be sure that the gradient corresponds to the scaled version instead of the unscaled version.
- `int64_t metropolis_check`: set to 1 to perform Metropolis checks, and 0 to skip Metropolis checks.
- `enum raytrace_integration_type itype`: the style of integration. This can be Leapfrog-style (use DKD for drift-kick-drift, KDK for kick-drift-kick, or RDKD to randomly choose between the two at each step), Omelyan 2nd-order (Omelyan), Omelyan 4th-order (Omelyan4th), or Forest-Ruth/Yoshida 4th-order (Yoshida).
- `enum raytrace_mcmc_type mcmc_type`: this can either be Raytracing for ray tracing or HMC for Hamiltonian Monte Carlo.

If you are running more than one sampler, you will also want to supply a non-default random state. You can do this via a call to the `Raytrace_Init_Random_State()` function:

```
void Raytrace_Init_Random_State(struct raytracer *r,
                               uint64_t *s, int64_t vals);
```

The parameters are as follows:

- `struct raytracer *r`: a pointer to a raytracer initialized via `Init_Raytracer()` as above.
- `uint64_t *s`: an array of 64-bit unsigned integers that represent the state vector. Internally, the code uses xoshiro256**, so 4 64-bit values are preferred, but the code can still make do with 1, 2, or 3 values.
- `int64_t vals`: the number of supplied 64-bit state values.

You can then run the ray tracing (or HMC) trajectory:

```
int64_t Raytrace(struct raytracer *r, double *likelihood);
```


This function integrates a single trajectory of the sampler, performs an optional Metropolis test, and returns the Metropolis result (1 for pass, 0 for fail). The sampler's new position is stored in an array of doubles in `r->x`. If the `likelihood` pointer is not `NULL`, the likelihood of the current position is stored at the pointer location.

All of the sampler parameters can be accessed and updated through the `raytracer` structure:

```
struct raytracer {
    double *x, *v, dt, refresh_rate;
    double *x0, *v0, *grad;
    int64_t DIMS, steps, metropolis_check;
    double (*likelihood)(double *, int64_t);
    void (*gradient)(double *, double *, int64_t);
    enum raytrace_integration_type itype;
    enum raytrace_mcmc_type mcmc_type;
    struct xoshiro256ss_state random_state;
};
```

Beyond the parameters already listed above in the initialization function, there are:

- `double *v`: the current momentum vector.
- `double *x0, *v0`: storage for original values of the position and momentum vectors. These arrays may not be allocated unless Metropolis checks have been enabled and a call to `Raytrace()` has been made.
- `double *grad`: the last computed value of the gradient vector.
- `struct xoshiro256ss_state random_state`: the internal random state vector.

To free the memory in the `raytracer` structure, you can use:

```
void Free_Raytracer(struct raytracer **r);
```

To call this, note that you should use `Free_Raytracer(&r)` for a `struct raytracer *` object `r`. This is because this function will reset `r` to a `NULL` pointer so that further use of the structure after its memory is freed will immediately trigger a memory fault instead of leading to mysterious bugs.

If you are using ray tracing to sample a neural network, you will want to initialize model parameters from a previously-optimized model (e.g., with Adam or similar), as ray tracing will not be as efficient at finding the model minimum as algorithms that are designed to minimize the loss as fast as possible. You will also want to adapt the likelihood scaling and the timestep size to your problem, as below.

Sampling will not give as low loss values as optimization, by definition. You can tune this difference by defining an appropriate likelihood that is a scaled version of your loss function. If D is the number of dimensions, a reasonable starting scaling is $L(x) = f_{\text{loss}}(x) * D/(2f_{\text{tol}})$, where f_{tol} is your desired loss tolerance. This choice scales the loss function (see the ray tracing paper) assuming that the effective number of constrained dimensions of the network is the same as the network parameter count. Since this is not always true, you may find that you need to decrease the scaling factor ($D/(2f_{\text{tol}})$), especially if the number of samples in your training set is less than the parameter count—but generally if your network is significantly more complex than the underlying function you are trying to fit. A strong sign that you need to decrease the scaling factor is if your validation loss is larger than your training loss — i.e., if overfitting is occurring.

The choice of timestep size depends on the geometry of the likelihood function and the stochasticity of the gradient, with higher stochasticity (i.e., lower mini-batch sizes) needing lower timestep sizes. For neural networks, a recommended starting timestep size is about $0.03/D^{0.5}$. Fortunately, it's relatively easy to tell if the timestep is too large or small. If the loss is diverging (which usually occurs within a few dozen steps after starting the chain), try repeatedly lowering the timestep by an order of magnitude until it no longer diverges. Then, increase the timestep size by factors of 2 until you reach divergence again, and use the largest timestep that does not result in divergence. Readers of your results will appreciate if you check for convergence by using a smaller timestep as well (e.g., 1/5 the size) and confirming that the posteriors that you find are consistent.

5 Examples

Examples for 10,000-dimensional Gaussian sampling for each implementation are in the `examples/` subdirectory, in the form of Python notebooks (for the JAX and PyTorch versions, and a Makefile and source file for the C version). The PyTorch version has an additional example to sample from the posterior distribution of a simple fully-connected neural network, taken from Section 3.2 of the ray tracing paper.

6 Autocorrelation Time Utility

If you don't already have a utility to estimate autocorrelation times, one is provided in the `util/` directory of the main source distribution (but not the pip packages, as this would complicate installation). To compile, type `make`. Usage is:

```
./acor_estimate <mcmc_output> <ndims> <nwalkers>
```

The input file (mcmc_output) should have one sample per line, with ndims space-separated parameters followed by the log-likelihood (or the chi-squared value, if preferred). If multiple walkers are used, walker outputs are expected to be interleaved as follows:

```
param1 param2 ... paramN log-prob #Walker 1, Sample 1
param1 param2 ... paramN log-prob #Walker 2, Sample 1
...
param1 param2 ... paramN log-prob #Walker W, Sample 1
param1 param2 ... paramN log-prob #Walker 1, Sample 2
param1 param2 ... paramN log-prob #Walker 2, Sample 2
...
param1 param2 ... paramN log-prob #Walker W, Sample 2
...
param1 param2 ... paramN log-prob #Walker 1, Sample K
param1 param2 ... paramN log-prob #Walker 2, Sample K
...
param1 param2 ... paramN log-prob #Walker W, Sample K
```

The estimator calculates the autocorrelation times of the parameter values, the absolute values of the parameter values, and the log-likelihood values, and reports the maximum. It uses the Sokal (1996) estimator for autocorrelation times, and prints a warning if any autocorrelation time (for any dimension) exceeds 20% of the total chain length—this is usually a sign that the samples are unconverged.